give correct sql code

Based on the images provided, I can see this is a SQL problem about analyzing driver delivery performance. The task requires calculating unsuccessful delivery rates for drivers by month, comparing with next working months, and computing percentage changes in rates. Here's the correct SQL solution:

Primary Solution Using Window Functions

```
SELECT
    driver_id,
    month,
    unsuccessful_rate,
    next_month_unsuccessful_rate,
    change_in_rate
FROM (
    SELECT
        d.driver_id,
        DATE_FORMAT(del.delivery_date, '%Y-%m') as month,
        ROUND(
            COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled') THEN 1 EN
            5
        ) as unsuccessful_rate,
        ROUND(
            LEAD(COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled') THEN
            OVER (PARTITION BY d.driver_id ORDER BY DATE_FORMAT(del.delivery_date, '%Y-%m
            5
        ) as next_month_unsuccessful_rate,
        ROUND(
            CASE
                WHEN LEAD(COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancell
                     OVER (PARTITION BY d.driver_id ORDER BY DATE_FORMAT(del.delivery_dat
                AND COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled') T
                THEN (
                    (LEAD(COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancell
                     OVER (PARTITION BY d.driver_id ORDER BY DATE_FORMAT(del.delivery_dat
                     - COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled'
                    / (COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled'
                ) * 100
                ELSE 0
            END,
            5
        ) as change_in_rate
    FROM Drivers d
    JOIN Deliveries del ON d.driver_id = del.driver_id
    GROUP BY d.driver_id, DATE_FORMAT(del.delivery_date, '%Y-%m')
) subquery
```

```
WHERE next_month_unsuccessful_rate IS NOT NULL
ORDER BY driver_id, month;
```

Alternative Solution Using CTEs

```
WITH monthly_stats AS (
    SELECT
        d.driver_id,
        DATE_FORMAT(del.delivery_date, '%Y-%m') as month,
        COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled') THEN 1 END) *
    FROM Drivers d
    JOIN Deliveries del ON d.driver_id = del.driver_id
    GROUP BY d.driver_id, DATE_FORMAT(del.delivery_date, '%Y-%m')
),
next_month_stats AS (
    SELECT
        ms1.driver_id,
        ms1.month,
        ms1.unsuccessful_rate,
        ms2.unsuccessful_rate as next_month_unsuccessful_rate
    FROM monthly_stats ms1
    LEFT JOIN monthly_stats ms2 ON ms1.driver_id = ms2.driver_id
        AND ms2.month = (
            SELECT MIN(month)
            FROM monthly_stats ms3
            WHERE ms3.driver_id = ms1.driver_id
            AND ms3.month > ms1.month
        )
)
SELECT
    driver_id,
    month,
    ROUND(unsuccessful_rate, 5) as unsuccessful_rate,
    ROUND(COALESCE(next_month_unsuccessful_rate, 0), 5) as next_month_unsuccessful_rate,
    ROUND(
        CASE
            WHEN next_month_unsuccessful_rate IS NOT NULL AND unsuccessful_rate > 0
            THEN ((next_month_unsuccessful_rate - unsuccessful_rate) / unsuccessful_rate)
            ELSE 0
        END,
        5
    ) as change_in_rate
FROM next_month_stats
WHERE next_month_unsuccessful_rate IS NOT NULL
ORDER BY driver_id, month;
```

Key Components

Unsuccessful Rate Calculation: Uses `COUNT(CASE WHEN del.delivery_status IN ('Undelivered', 'Cancelled') THEN 1 END) * 100.0 / COUNT(*)` to calculate the percentage of unsuccessful deliveries. [1] [2]

Next Month Comparison: The `LEAD()` window function retrieves the unsuccessful rate from the next chronological month for each driver.[3] [4] [5]

Percentage Change: Calculates the percentage change using the formula `((next_month_rate - current_rate) / current_rate) * 100` when the current rate is greater than zero.[6] [7] [8]

Filtering and Ordering: Only includes drivers with next working months and orders results by `driver_id` and `month` as specified.[2] [1]

The first solution using window functions is generally more efficient for this type of time-series analysis, while the CTE approach provides better readability and easier debugging.[9] [10] [11]

⁂

1. https://stackoverflow.com/questions/48240474/performance-issue-on-query-with-math-calculations
2. https://www.devtodev.com/resources/articles/sql-mistakes-to-avoid-a-guide-to-optimizing-your-queries
3. https://www.w3resource.com/mysql-exercises/mysql-query-to-find-the-lead-and-lag-values-for-sales-using-a-window-function.php
4. https://datalemur.com/sql-tutorial/sql-time-series-window-function-lead-lag
5. https://www.thedataschool.com.au/daniel-lawson/lag-lead-advanced-sql-window-functions/
6. https://stackoverflow.com/questions/62067049/calculating-percent-change-month-over-month-in-sql
7. http://www.silota.com/docs/recipes/sql-mom-growth-rate.html
8. https://www.linkedin.com/pulse/sql-challenge-month-over-month-revenue-growth-muad-rahman-rl4sf
9. https://www.coginiti.co/tutorials/intermediate/sql-window-functions/
10. https://blog.devops.dev/sql-queries-for-time-series-data-2f6ce2680689
11. https://www.dataquest.io/tutorial/window-functions-in-sql-tutorial/
12. image.jpg
13. image.jpg
14. image.jpg
15. image.jpg
16. image.jpg
17. image.jpg
18. https://kleene.ai/sql-for-data-analysis-2/
19. https://www.alexanderjarvis.com/what-is-sql-to-customer-conversion-rate-in-saas/
20. https://moldstud.com/articles/p-improving-application-performance-through-successful-case-studies-highlighting-the-efficiency-gains-of-sql-clr
21. https://stackoverflow.com/questions/64085310/calculate-failure-rate-per-city-in-orders-table
22. https://www.youtube.com/watch?v=fJpzn8nDCnw
23. https://mode.com/sql-tutorial/sql-window-functions/
24. https://trailhead.salesforce.com/content/learn/modules/customer-data-platform-insights-using-sql/review-sql-examples
25. https://www.sqlservercentral.com/forums/topic/percentage-of-total-trips-broken-down-by-month

26. https://dora.dev/guides/dora-metrics-four-keys/